

# Algorithms

## Chapter 3

With Question/Answer Animations



# Algorithms

Abu Ja'far Mohammed Ibin Musa Al-Khowarizmi  
(780-850)

**Definition:** An *algorithm* is a finite set of precise instructions for performing a computation or for solving a problem.

## Properties of Algorithms:

- *Input*
- *Output*
- *Correctness*
- *Finiteness*
- *Effectiveness*
- *Generality*

# Finding the Maximum Element in a Finite Sequence

- The algorithm in pseudocode:

```
procedure max( $a_1, a_2, \dots, a_n$ : integers)  
   $max := a_1$   
  for  $i := 2$  to  $n$   
    if  $max < a_i$  then  $max := a_i$   
  return  $max$ { $max$  is the largest element}
```

- Does this algorithm have all the properties listed on the previous slide?

# Some Example Algorithm Problems

● Three classes of problems will be studied in this section.

1. *Searching Problems*: finding the position of a particular element in a list.
2. *Sorting problems*: putting the elements of a list into increasing order.
3. *Optimization Problems*: determining the optimal value (maximum or minimum) of a particular quantity over all possible inputs.

# Linear Search Algorithm

- The linear search algorithm locates an item in a list by examining elements in the sequence one at a time, starting at the beginning.
- First compare  $x$  with  $a_1$ . If they are equal, return the position 1.
- If not, try  $a_2$ . If  $x = a_2$ , return the position 2.
- Keep going, and if no match is found when the entire list is scanned, return 0.

```
procedure linear search( $x$ :integer,  
                         $a_1, a_2, \dots, a_n$ : distinct integers)  
 $i := 1$   
while ( $i \leq n$  and  $x \neq a_i$ )  
     $i := i + 1$   
if  $i \leq n$  then  $location := i$   
else  $location := 0$   
return  $location$  { $location$  is the subscript of the term that  
                  equals  $x$ , or is 0 if  $x$  is not found}
```

# Binary Search

- Assume the input is a list of items **in increasing order**.
- The algorithm begins by comparing the element to be found with the middle element.
  - If the middle element is lower, the search proceeds with the upper half of the list.
  - If it is not lower, the search proceeds with the lower half of the list (through the middle position).
- Repeat this process until we have a list of size 1.
  - If the element we are looking for is equal to the element in the list, the position is returned.
  - Otherwise, 0 is returned to indicate that the element was not found.
- In Section 3.3, we show that the binary search algorithm is much more efficient than linear search.

# Binary Search

- Here is a description of the binary search algorithm in pseudocode.

```
procedure binary search( $x$ : integer,  $a_1, a_2, \dots, a_n$ : increasing integers)
   $i := 1$  { $i$  is the left endpoint of interval}
   $j := n$  { $j$  is right endpoint of interval}
  while  $i < j$ 
     $m := \lfloor (i + j) / 2 \rfloor$ 
    if  $x > a_m$  then  $i := m + 1$ 
    else  $j := m$ 
  if  $x = a_i$  then  $location := i$ 
  else  $location := 0$ 
  return  $location$  { $location$  is the subscript  $i$  of the term  $a_i$  equal to  $x$ ,
    or 0 if  $x$  is not found}
```

# Binary Search

**Example:** The steps taken by a binary search for 19 in the list:

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

1. The list has 16 elements, so the midpoint is 8. The value in the 8<sup>th</sup> position is 10. Since  $19 > 10$ , further search is restricted to positions 9 through 16.

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

2. The midpoint of the list (positions 9 through 16) is now the 12<sup>th</sup> position with a value of 16. Since  $19 > 16$ , further search is restricted to the 13<sup>th</sup> position and above.

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

3. The midpoint of the current list is now the 14<sup>th</sup> position with a value of 19. Since  $19 \neq 19$ , further search is restricted to the portion from the 13<sup>th</sup> through the 14<sup>th</sup> positions.

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

4. The midpoint of the current list is now the 13<sup>th</sup> position with a value of 18. Since  $19 > 18$ , search is restricted to the portion from the 14<sup>th</sup> position through the 14<sup>th</sup>.

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

5. Now the list has a single element and the loop ends. Since  $19 = 19$ , the location 14 is returned.

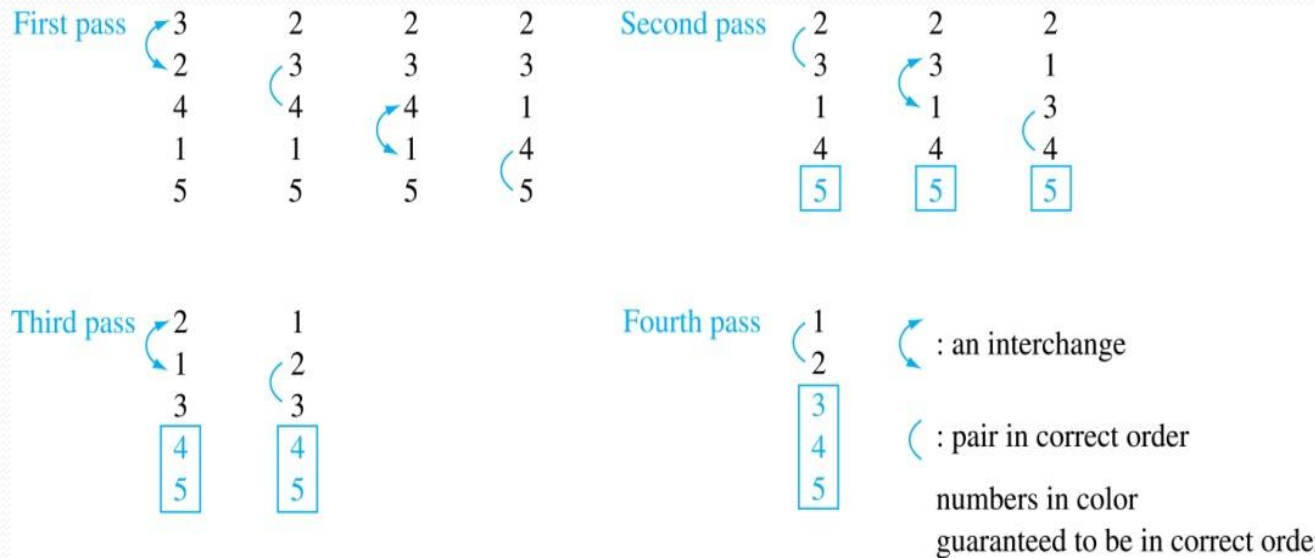
# Bubble Sort

- *Bubble sort* makes multiple passes through a list. Every pair of elements that are found to be out of order are interchanged.

```
procedure bubblesort( $a_1, \dots, a_n$ : real numbers  
                    with  $n \geq 2$ )  
    for  $i := 1$  to  $n - 1$   
        for  $j := 1$  to  $n - i$   
            if  $a_j > a_{j+1}$  then interchange  $a_j$  and  $a_{j+1}$   
    { $a_1, \dots, a_n$  is now in increasing order}
```

# Bubble Sort

**Example:** Show the steps of bubble sort with 3 2 4 1 5



- At the first pass the largest element has been put into the correct position
- At the end of the second pass, the 2<sup>nd</sup> largest element has been put into the correct position.
- In each subsequent pass, an additional element is put in the correct position.

# Big-O Notation

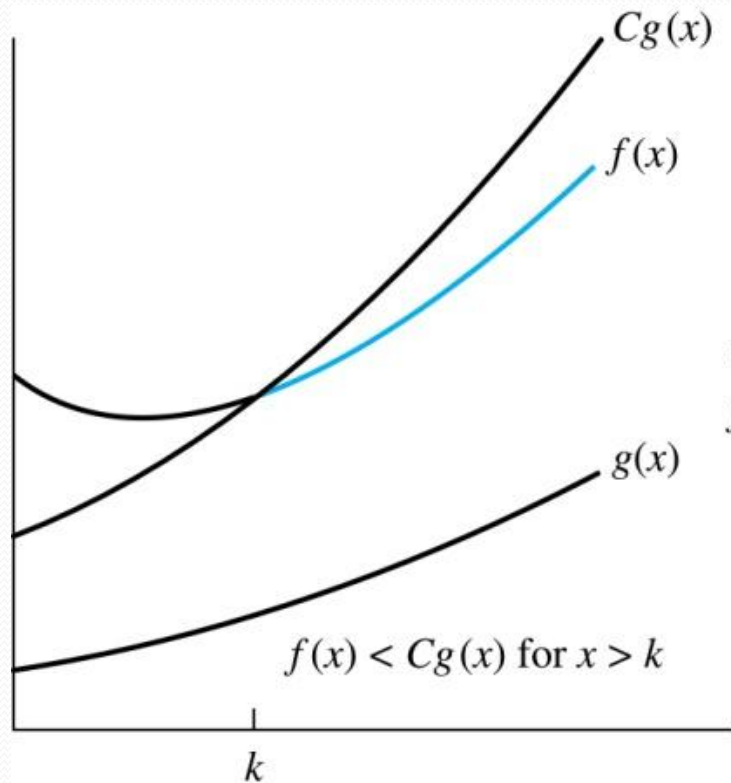
**Definition:** Let  $f$  and  $g$  be functions from the set of integers or the set of real numbers to the set of real numbers. We say that  $f(x)$  is  $O(g(x))$  if there are constants  $C$  and  $k$  such that

$$|f(x)| \leq C|g(x)|$$

whenever  $x > k$ . (illustration on next slide)

- This is read as “ $f(x)$  is big- $O$  of  $g(x)$ ” or “ $g$  asymptotically dominates  $f$ .”
- The constants  $C$  and  $k$  are called *witnesses* to the relationship  $f(x)$  is  $O(g(x))$ .
- Only one pair of witnesses is needed.

# Illustration of Big-O Notation



$$f(x) \text{ is } O(g(x))$$

The part of the graph of  $f(x)$  that satisfies  $f(x) < Cg(x)$  is shown in color.

$$f(x) < Cg(x) \text{ for } x > k$$

# Using the Definition of Big-O Notation

**Example:** Show that  $f(x) = x^2 + 2x + 1$  is  $O(x^2)$  .

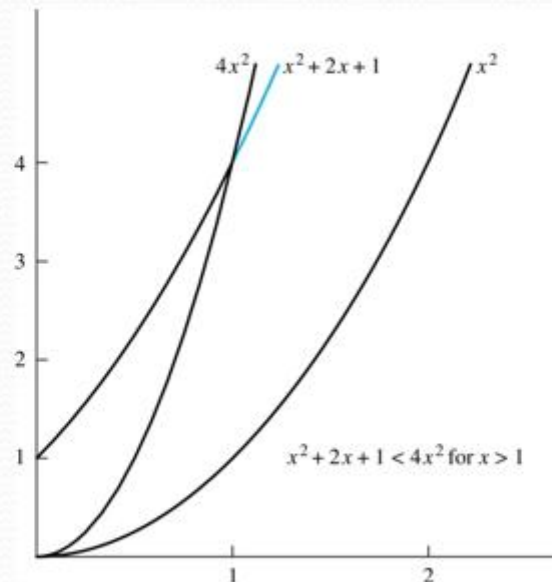
**Solution:** Since when  $x > 1$ ,  $x < x^2$  and  $1 < x^2$

$$0 \leq x^2 + 2x + 1 \leq x^2 + 2x^2 + x^2 = 4x^2$$

● Can take  $C = 4$  and  $k = 1$  as witnesses to show that

# Illustration of Big-O Notation

$$f(x) = x^2 + 2x + 1 \text{ is } O(x^2)$$



# Using the Definition of Big-O Notation

**Example:** Show that  $7x^2$  is  $O(x^3)$ .

**Solution:** When  $x > 7$ ,  $7x^2 < x^3$ . Take  $C = 1$  and  $k = 7$  as witnesses to establish that  $7x^2$  is  $O(x^3)$ .

(Would  $C = 7$  and  $k = 1$  work?)

**Example:** Show that  $n^2$  is not  $O(n)$ .

**Solution:** Suppose there are constants  $C$  and  $k$  for which  $n^2 \leq Cn$ , whenever  $n > k$ . Then (by dividing both sides of  $n^2 \leq Cn$ ) by  $n$ , then  $n \leq C$  must hold for all  $n > k$ . A contradiction!

# Big- $O$ Estimates for Polynomials

**Example:** Let  $f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$   
where  $a_0, a_1, \dots, a_n$  are real numbers with  $a_n \neq 0$ .  
Then  $f(x)$  is  $O(x^n)$ .

# Big- $O$ Estimates for some Important Functions

**Example:** Use big- $O$  notation to estimate the sum of the first  $n$  positive integers.

**Solution:**  $1 + 2 + \cdots + n \leq n + n + \cdots + n = n^2$

$1 + 2 + \cdots + n$  is  $O(n^2)$  taking  $C = 1$  and  $k = 1$ .

**Example:** Use big- $O$  notation to estimate the factorial function  $f(n) = n! = 1 \times 2 \times \cdots \times n$ .

**Solution:**

$$n! = 1 \times 2 \times \cdots \times n \leq n \times n \times \cdots \times n = n^n$$

$n!$  is  $O(n^n)$  taking  $C = 1$  and  $k = 1$ .

# Big- $O$ Estimates for some Important Functions

**Example:** Use big- $O$  notation to estimate  $\log n!$

**Solution:** Given that  $n! \leq n^n$  (previous slide)

then  $\log(n!) \leq n \cdot \log(n)$ .

Hence,  $\log(n!)$  is  $O(n \cdot \log(n))$  taking  $C = 1$  and  $k = 1$ .

- big- $O$  of any constant is  $O(1)$ .

# Combinations of Functions

- If  $f_1(x)$  is  $O(g_1(x))$  and  $f_2(x)$  is  $O(g_2(x))$  then  $(f_1 + f_2)(x)$  is  $O(\max(|g_1(x)|, |g_2(x)|))$ .
- If  $f_1(x)$  is  $O(g_1(x))$  and  $f_2(x)$  is  $O(g_2(x))$  then  $(f_1 f_2)(x)$  is  $O(g_1(x)g_2(x))$ .

**Example:** Give a big- $O$  estimate for  $f(n) = 3n\log(n!) + (n^2 + 3)\log n$ , where  $n$  is a positive integer.

Solution:

$$3n = O(n)$$

$$\log(n!) = O(n\log n)$$

$$\therefore 3n\log(n!) = O(n (n\log n)) = O(n^2\log n)$$

$$(n^2 + 3) = O(n^2)$$

$$\log n = O(\log n)$$

$$\therefore (n^2 + 3)\log n = O(n^2 (\log n)) = O(n^2\log n)$$

$$\begin{aligned}\therefore f(n) &= 3n\log(n!) + (n^2 + 3)\log n \\ &= O(\max(n^2\log n, n^2\log n)) = O(n^2\log n)\end{aligned}$$

**Example:** Give big- $O$  estimate for  $f(x) = (x + 1)\log(x^2 + 1) + 3x^2$

Solution:

$$(x + 1) = O(x)$$

$$\begin{aligned}\log(x^2 + 1) &\leq \log(x^2 + x^2) \text{ [when } x \geq 1\text{]} \\ &= \log(2x^2) \\ &= \log 2 + \log x^2 \\ &= \log 2 + 2\log x \\ &\leq \log x + 2\log x \text{ [when } x \geq 2\text{]} \\ &= 3\log x\end{aligned}$$

$$\therefore \log(x^2 + 1) = O(\log x) \text{ [} k = 2 \text{ and } C = 3\text{]}$$

$$\therefore (x + 1)\log(x^2 + 1) = O(x\log x)$$

$$3x^2 = O(x^2)$$

$$\therefore f(x) = (x + 1)\log(x^2 + 1) + 3x^2 = O(\max(x\log x, x^2)) = O(x^2)$$

# Big-Omega Notation

**Definition:** Let  $f$  and  $g$  be functions from the set of integers or the set of real numbers to the set of real numbers. We say that  $f(x)$  is  $\Omega(g(x))$  if there are constants  $C$  and  $k$  such that

$$|f(x)| \geq C|g(x)| \text{ when } x > k.$$

- Big-O gives an upper bound on the growth of a function, while Big-Omega gives a lower bound.
- $f(x)$  is  $\Omega(g(x))$  if and only if  $g(x)$  is  $O(f(x))$ . This follows from the definitions.

# Big-Omega Notation

**Example:** Show that  $f(x) = 8x^3 + 5x^2 + 7$  is  $\Omega(g(x))$  where  $g(x) = x^3$ .

**Solution:**  $f(x) = 8x^3 + 5x^2 + 7 \geq 8x^3$  for all positive real numbers  $x$ . (for  $k=0$ ,  $c=8$ )

● Is it also the case that  $g(x) = x^3$  is  $O(8x^3 + 5x^2 + 7)$  ?

# Big-Theta Notation

- **Definition:** Let  $f$  and  $g$  be functions from the set of integers or the set of real numbers to the set of real numbers. The function  $f(x)$  is  $\Theta(g(x))$  if  $f(x)$  is  $O(g(x))$  and  $f(x)$  is  $\Omega(g(x))$
- $f(x)$  is  $\Theta(g(x))$  if and only if there exists constants  $C_1$ ,  $C_2$  and  $k$  such that  $C_1g(x) < f(x) < C_2g(x)$  if  $x > k$ . This follows from the definitions of big-O and big-Omega.

# Big-Theta Notation

**Example:** Show that  $f(x) = 3x^2 + 8x \log x$  is  $\Theta(x^2)$ .

**Solution:**

- $3x^2 + 8x \log x \leq 11x^2$  for  $x > 1$ ,  
since  $0 \leq 8x \log x \leq 8x^2$ .
- Hence,  $3x^2 + 8x \log x$  is  $O(x^2)$ .
- $x^2$  is clearly  $O(3x^2 + 8x \log x)$
- Hence,  $3x^2 + 8x \log x$  is  $\Theta(x^2)$ .

# Big-Theta Notation

- When  $f(x)$  is  $\Theta(g(x))$  it must also be the case that  $g(x)$  is  $\Theta(f(x))$ .
- Note that  $f(x)$  is  $\Theta(g(x))$  if and only if it is the case that  $f(x)$  is  $O(g(x))$  and  $g(x)$  is  $O(f(x))$ .

# Big-Theta Estimates for Polynomials

**Theorem:** Let  $f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$  where  $a_0, a_1, \dots, a_n$  are real numbers with  $a_n \neq 0$ . Then  $f(x)$  is of order  $x^n$  (or  $\Theta(x^n)$ ).

**Example:**

The polynomial  $f(x) = 8x^5 + 5x^2 + 10$  is order of  $x^5$  (or  $\Theta(x^5)$ ).

The polynomial  $f(x) = 8x^{199} + 7x^{100} + x^{99} + 5x^2 + 25$  is order of  $x^{199}$  (or  $\Theta(x^{199})$ ).

**Question:** Show that the sum of the first  $n$  positive integers is  $\Theta(n^2)$ .

# Complexity Analysis of Algorithms

**Example:** Describe the time complexity of the algorithm for finding the maximum element in a finite sequence.

```
procedure max( $a_1, a_2, \dots, a_n$ : integers)
   $max := a_1$ 
  for  $i := 2$  to  $n$ 
    if  $max < a_i$  then  $max := a_i$ 
  return  $max$ { $max$  is the largest element}
```

**Solution:** Count the number of comparisons.

- The  $max < a_i$  comparison is made  $n - 1$  times.
- Each time  $i$  is incremented, a test is made to see if  $i \leq n$ .
- One last comparison determines that  $i > n$ .
- Exactly  $2(n - 1) + 1 = 2n - 1$  comparisons are made.

Hence, the time complexity of the algorithm is  $\Theta(n)$ .

# Worst-Case Complexity of Linear Search

**Example:** Determine the time complexity of the linear search algorithm.

```
procedure linear search(x:integer,  
                        a1, a2, ..., an: distinct integers)  
i := 1  
while (i ≤ n and x ≠ ai)  
    i := i + 1  
if i ≤ n then location := i  
else location := 0  
return location{location is the subscript of the term that equals x, or is 0 if  
                  x is not found}
```

**Solution:** Count the number of comparisons.

- At each step two comparisons are made;  $i \leq n$  and  $x \neq a_i$ .
- To end the loop, one comparison  $i \leq n$  is made.
- After the loop, one more  $i \leq n$  comparison is made.

If  $x = a_i$ ,  $2i + 1$  comparisons are used. If  $x$  is not on the list,  $2n + 1$  comparisons are made and then an additional comparison is used to exit the loop. So, in the worst case  $2n + 2$  comparisons are made. Hence, the complexity is  $\Theta(n)$ .

# Average-Case Complexity of Linear Search

**Example:** Describe the average case performance of the linear search algorithm. (Although usually it is very difficult to determine average-case complexity, it is easy for linear search.)

**Solution:** Assume the element is in the list and that the possible positions are equally likely. By the argument on the previous slide, if  $x = a_i$ , the number of comparisons is

$$2i + 1. \quad \frac{3+5+7+\dots+(2n+1)}{n} = \frac{2(1+2+3+\dots+n)+n}{n} = \frac{2\left[\frac{n(n+1)}{2}\right]}{n} + 1 = n + 2$$

Hence, the average-case complexity of linear search is  $\Theta(n)$ .

# Worst-Case Complexity of Binary Search

**Example:** Describe the time complexity of binary search in terms of the number of comparisons used.

```
procedure binary search( $x$ : integer,  $a_1, a_2, \dots, a_n$ : increasing integers)
 $i := 1$  { $i$  is the left endpoint of interval}
 $j := n$  { $j$  is right endpoint of interval}
while  $i < j$ 
     $m := \lfloor (i + j) / 2 \rfloor$ 
    if  $x > a_m$  then  $i := m + 1$ 
    else  $j := m$ 
if  $x = a_i$  then  $location := i$ 
else  $location := 0$ 
return  $location$  { $location$  is the subscript  $i$  of the term  $a_i$  equal to  $x$ , or 0 if  $x$  is not found}
```

**Solution:** Assume (for simplicity)  $n = 2^k$  elements. Note that  $k = \log n$ .

- Two comparisons are made at each stage;  $i < j$ , and  $x > a_m$ .
- At the first iteration the size of the list is  $2^k$  and after the first iteration it is  $2^{k-1}$ . Then  $2^{k-2}$  and so on until the size of the list is  $2^1 = 2$ .
- At the last step, a comparison tells us that the size of the list is the size is  $2^0 = 1$  and the element is compared with the single remaining element.
- Hence, at most  $2k + 2 = 2 \log n + 2$  comparisons are made.
- Therefore, the time complexity is  $\Theta(\log n)$ , better than linear search.

# Worst-Case Complexity of Bubble Sort

**Example:** What is the worst-case complexity of bubble sort in terms of the number of comparisons made?

```
procedure bubblesort( $a_1, \dots, a_n$ : real numbers  
                    with  $n \geq 2$ )  
  for  $i := 1$  to  $n - 1$   
    for  $j := 1$  to  $n - i$   
      if  $a_j > a_{j+1}$  then interchange  $a_j$  and  $a_{j+1}$   
  { $a_1, \dots, a_n$  is now in increasing order}
```

**Solution:** A sequence of  $n-1$  passes is made through the list. On each pass  $n - i$  comparisons are made.

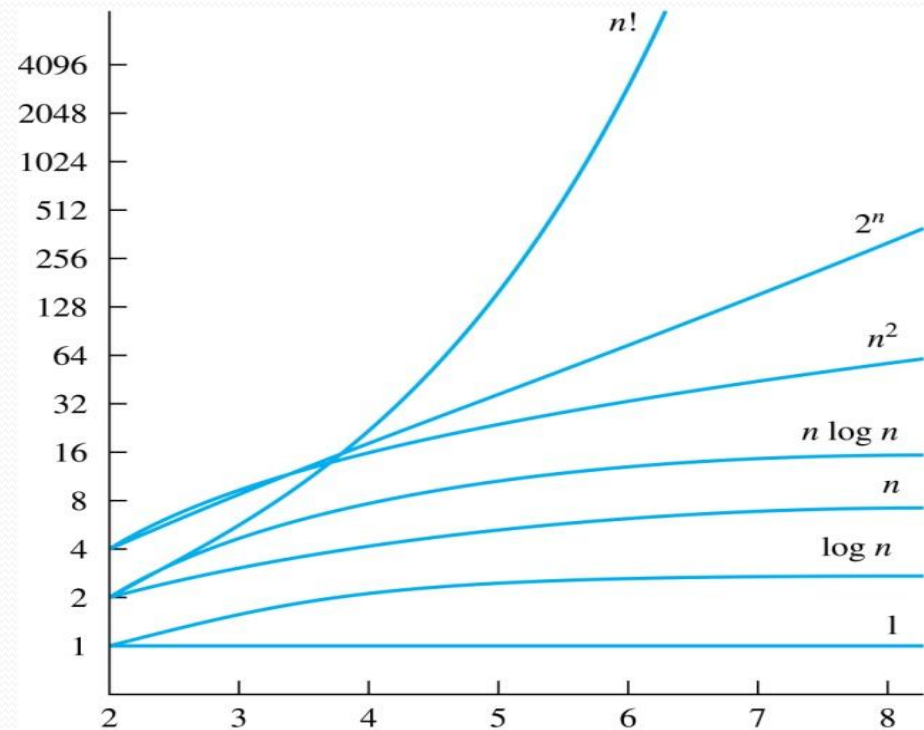
$$(n - 1) + (n - 2) + \dots + 2 + 1 = \frac{n(n-1)}{2}$$

The worst-case complexity of bubble sort is  $\Theta(n^2)$  since  $\frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$ .

# Understanding the Complexity of Algorithms

**TABLE 1** Commonly Used Terminology for the Complexity of Algorithms.

| <i>Complexity</i>             | <i>Terminology</i>      |
|-------------------------------|-------------------------|
| $\Theta(1)$                   | Constant complexity     |
| $\Theta(\log n)$              | Logarithmic complexity  |
| $\Theta(n)$                   | Linear complexity       |
| $\Theta(n \log n)$            | Linearithmic complexity |
| $\Theta(n^b)$                 | Polynomial complexity   |
| $\Theta(b^n)$ , where $b > 1$ | Exponential complexity  |
| $\Theta(n!)$                  | Factorial complexity    |



**Note the difference in behavior of functions as  $n$  gets larger**

# Understanding the Complexity of Algorithms

**TABLE 2** The Computer Time Used by Algorithms.

| <i>Problem Size</i> | <i>Bit Operations Used</i> |              |                       |                      |                       |                      |
|---------------------|----------------------------|--------------|-----------------------|----------------------|-----------------------|----------------------|
| <i>n</i>            | <i>log n</i>               | <i>n</i>     | <i>n log n</i>        | <i>n<sup>2</sup></i> | <i>2<sup>n</sup></i>  | <i>n!</i>            |
| 10                  | $3 \times 10^{-11}$ s      | $10^{-10}$ s | $3 \times 10^{-10}$ s | $10^{-9}$ s          | $10^{-8}$ s           | $3 \times 10^{-7}$ s |
| $10^2$              | $7 \times 10^{-11}$ s      | $10^{-9}$ s  | $7 \times 10^{-9}$ s  | $10^{-7}$ s          | $4 \times 10^{11}$ yr | *                    |
| $10^3$              | $1.0 \times 10^{-10}$ s    | $10^{-8}$ s  | $1 \times 10^{-7}$ s  | $10^{-5}$ s          | *                     | *                    |
| $10^4$              | $1.3 \times 10^{-10}$ s    | $10^{-7}$ s  | $1 \times 10^{-6}$ s  | $10^{-3}$ s          | *                     | *                    |
| $10^5$              | $1.7 \times 10^{-10}$ s    | $10^{-6}$ s  | $2 \times 10^{-5}$ s  | 0.1 s                | *                     | *                    |
| $10^6$              | $2 \times 10^{-10}$ s      | $10^{-5}$ s  | $2 \times 10^{-4}$ s  | 0.17 min             | *                     | *                    |

This is assuming that each bit operation takes  $10^{-11}$  seconds.

Times of more than  $10^{100}$  years are indicated with an \*.

# Review Questions

- Give a *big - O* estimate for each of these functions. For the function  $g$  in your estimate that  $f(x)$  is  $O(g(x))$ , use a simple function  $g$  of the smallest order.

**a)**  $n \log(n^2 + 1) + n^2 \log n$

**b)**  $(n \log n + 1)^2 + (\log n + 1)(n^2 + 1)$

**c)**  $n^{2^n} + n^{n^2}$

- Show that  $3x^2 + x + 1$  is  $\Theta(3x^2)$  by directly finding the constants  $k, C1$ , and  $C2$
- Give a *big - O* estimate for the number of operations, where an operation is a comparison or a multiplication, used in this segment of an algorithm where  $a_1, a_2, \dots, a_n$  are positive real numbers.

$m := 0$

**for**  $i := 1$  **to**  $n$

**for**  $j := i + 1$  **to**  $n$

$m := \max(a_i a_j, m)$

- Describe the linear search and binary search algorithm for finding an integer in a list of integers in increasing order. Compare the worst-case time complexities of these two algorithms. Is one of these algorithms always faster than the other?